std::zstring_view

Document #	P3655R1
Date	2025-05-18
Targeted subgroups	LEWG
Ship vehicle	C++29
Reply-to	Peter Bindels <dascandy@gmail.com> Hana Dusíková <hanicka@hanicka.net> Jeremy Rifkin <jeremy@rifkin.dev>

§ 1 Abstract

We propose a standard string view type that guarantees null-termination.

§ 2 Introduction

C++17 introduced `std::string_view`, a non-owning view of a continuous sequence of characters. It is cheap to use, offers fast operations, and replaced most uses of `const std::string&` or `const char*` as function parameters. The utility and interface of string views as well as the benefits of not having to do unnecessary `strlen` calculations on C-style strings makes string view types highly desirable for working with strings in C++.

Unlike `const std::string&` and many but not all `const char*`, `std::string_view` is not null-terminated. This allows it to have fast and cheap substring operations. However, this also means `std::string_view` is not a suitable replacement for either of the two aforementioned types whenever a null-terminated C-style string is needed. While most C++ code mostly interfaces with C++ code, it is not uncommon to need to use operating system calls, C interfaces, third-party library APIs, or even C++ standard library APIs which require null-terminated strings. Because of a lack of a desirable option for passing non-owned null-terminated strings, `std::string_view` parameters are none the less sometimes used today in cases where null-terminated strings are needed, calling `std::string_view::data` to get a `const char*`. This is, needless to say, very bug-prone.

For this reason, many C++ developers use custom `zstring_view` or `cstring_view` types which are guaranteed to be null-terminated. We propose standardizing this utility.

§ 3 Previous Papers

The idea of `zstring_view` was present even in the original paper for `string_view` (Sept 2012 - Feb 2014) https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2014/n3921.html#null-termination:

> Another option would be to define a separate zstring_view class to represent null-terminated strings and let it decay to string_view when necessary. That's plausible but not part of this proposal.

An attempt was made in Feb 2019 to concretely propose the type (renamed to `cstring_view`) with http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1402r0.pdf, but it failed to gain consensus in LEWGI at https://wiki.edg.com/bin/view/Wg21kona2019/P1402. In fact, the discussion there concluded with "CONSENSUS: We will not pursue P1402R0 or this problem space." It is also worth noting that contracts were briefly contemplated in the minutes as a way of addressing the problem, however, contracts can't check for a null character safely as it wouldn't be part of the view. Similarly, other runtime checks for `string_view` null-termination are off the table.

However, in 2024 https://wg21.link/p3081 was published which contains an aside regarding a null-terminated `zstring_view` in section 8, noting:

> This is one of the commonly-requested features from the https://github.com/microsoft/GSL library that does not yet have a std:: equivalent. It was specifically requested by several reviewers of this work.

During the discussion, it was made clear that `zstring_view` deserves attention on its own and it should not be happenstance introduced as part of an entirely different topic. While P3081 did not end up passing for reasons other than `zstring_view`, it is another example of the utility being seen as desirable.

We also have https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p2996r9.html, which in its specification of functions returning a `string_view` or `u8string_view` tries its best to say it's actually supposed to be a `zstring_view`, except without naming the type for existential reasons:

> Any function in namespace std::meta that whose return type is string_view or u8string_view returns an object V such that `V.data()[V.size()] == '\0'`.

As such, we do believe that there is both space for such a type, and a desire from multiple angles to have it defined in the standard library.

§ 4 Reasons to have the type

Many functions right now whether C++ standard library calls, operating system calls, or third-party library calls require null-terminated C-style strings. Absent an efficient type that can represent a non-owning view of a null-terminated string, these functions must take either a `const char*` and incur potential `strlen` overhead, take a `const std::string&` and possibly superfluously copy and allocate, take a `string_view` and make a copy, or haphazardly take a `std::string_view` with a fragile unenforceable contract that it is a view of a null-terminated string. Such a contract would be unenforceable because `pre(sv[sv.size()] == 0)` is potentially undefined behavior.

String views tend to start their life coming from a string literal or from a type that owns its internal buffer, often `std::string`. Both of these are places that can always create a `zstring_view` instead, as they both know their data is inherently null terminated.

Strings are very often just passed along with a non-owning string type all the way to where they are used as data. Some of these potential uses, such as calling `std::ofstream::write`, do not rely on the null terminator, but others, like `std::ofstream::ofstream`, do. Using `string_view` as the intermediate type loses the knowledge that a null terminator is already guaranteed to be present, requiring the developer to either make assumptions or make a copy. Another common use might be creating a stringstream from a string https://stackoverflow.com/questions/58524805/is-there-a-way-to-create-a-stringstream-from-a-string-view-without-copying-data.

We do not have to look far for examples of `std::string_view` being used in bug-prone ways with APIs expecting null-terminators. Searching `/\.data\(\)/ string_view language:c++ -is:fork` on GitHub code search turns up two examples on the first page:

From https://github.com/surge-synthesizer/shortcircuit-xt/blob/ba6ea3a2e703e4fa0ed069427aa42b668699b624/libs/md5sum/demo.cc#L37

```
std::optional<std::string> hash_file(const std::string_view &file_name) {
    auto fd = open(file_name.data(), O_RDONLY);
    ...
```

From https://github.com/VisualGMQ/gmq_header/blob/cbc2853f391acc51ddd25a50d567cac404776413/log.hpp#L66

```
void log(Level level, std::string_view funcName, std::string_view filename, unsigned int line, Args&&... arg
    if (level <= level_) {
        printf("[%s][%s][%s][%u]", Level2Str(level).data(), filename.data(), funcName.data(), line);
        ...
```

These two examples could be argued to be bad code or misuses of `std::string_view`, however, the fact that they are written reflects the desire for the ability to use string view types in such cases.

These problems are visible enough that we have targeted patches for specific instances of this problem in the standard - https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2495r0.pdf attempts to directly patch this specific example. The solution bypasses that the problem is that we're missing an unbroken type-safe chain of knowledge that the value being passed is or is not null-terminated.

The type effectively fills out the design space that exists around strings within C++. We started off with `std::string` as an owning string type, ambiguously being specified as having a null terminator or not, clarified in C++11 to definitely *have* a null terminator. C++14 added `std::string_view` to that set, offering a non-null-terminated non-owning string type. The type itself raises the question, should we add a non-null-terminated owning string type, and/or a null-terminated non-owning string type? We're proposing the last of these, leaving only the non-null-terminated owning string type as not present. We believe there is no situation to be written where a non-null-terminated owning string type would have a measurable benefit over the null-terminated owning string type that we have, and as such it is not worth the added complexity.

Without them we retain the question we had before - `const char*`, `const std::string&` or `string_view`? The first loses lots of type safety and potentially requires redundant `strlen` computation elsewhere in the software, the second requires it to be a `std::string` but retains null-termination knowledge, the last offers the ability to send any string type constructible to a string_view, but loses any knowledge of null termination. For the common case of passing along "some string input" to a function that ends up requiring null termination, the proposed `zstring_view` is the only type that properly captures it.

Since the first draft of this paper the use of `cstring_view` and `zstring_view` on Github has grown from 1.9k to 2.1k, indicating active use, and in many cases people adding new implementations of the type. Many contain the subtle bugs that we highlight in this paper, illustrating why it is a good idea to add the type to the standard.

5 Reasons not to have the type

In an ideal world, we could actually fix the operating systems and third-party libraries to accept pointer-and-size strings in all places, removing the need for null termination to exist, and for null termination propagation to be relevant. Sometimes, in particular from environments where this may not be unrealistic in a subset, the argument is voiced that `zstring_view` does not offer any benefits.

Adding the type complicates the type system around strings by having more options for string types. This is not as much of an argument as it seems, since for each site a single type is the most appropriate, and the only places that would differ are those where the difference can make a meaningful performance impact - ie, the exact kind of tight loop where the type is useful for being able to make the difference.

6 Subtle details

6.1 zstring_view and string_view relation

Initially we expected that `zstring_view` may be implementable in terms of `string_view`. This is not actually true; in particular it is not well-formed to use `string_view`'s `operator=` to assign a non-null-terminated `string_view` to a `zstring_view`. As such, there can not be an inheritance relation between the two.

`zstring_view` is transparently convertible to a `string_view`. For the conversion operator, there is no `operator=` that could affect the original `zstring_view`, and as such it is a correct change to do. It in particular makes it so that any use of `zstring_view` in places where string_view is expected or desired will work without additional work.

This brings into reality an overload ambiguity:

```
void handle(string_view v);
void handle(zstring_view v);
handle("Hello World!");
```

This is now an ambiguous function call. It is not a new problem; https://godbolt.org/z/c31e31fKW shows that the exact same problem already happens with regular `std::string` and `std::string_view`. The reason is somewhat fundamental; all three types model the concept "string" and *are* ambiguous. The user should be clear about which properties of string it's expecting. Adding this new type only adds to the vocabulary the option to say "non-owning null-terminated", giving the user better choices rather than complicating it.

6.1.1 Deprioritizing one overload

When one overload is preferred, but others should exist, the less preferred ones can be marked with `requires true`. This lifts one overload up out of the overload set for ambiguous matches, while leaving the rest to exist.

6.1.2 Tag type forced conversions for exact matches

https://godbolt.org/z/8qssnq8rf and https://godbolt.org/z/xG7955hf9 illustrate the idea of using a wrapping template to make one overload stand out.

6.1.3 Explicitly adding a function to disambiguate manually

https://godbolt.org/z/9zoGEh1cv demonstrates adding an overload that matches anything that isn't an exact match, which indirects to a function that hand-picks an overload.

6.2 Construction

`std::basic_string_view` offers a handful of constructors:

```
constexpr basic_string_view(const charT* str); // (1)
constexpr basic_string_view(const charT* str, size_type len); // (2)
template<class It, class End>
constexpr basic_string_view(It begin, End end); // (3)
template<class R>
constexpr explicit basic_string_view(R&& r); // (4)
basic_string_view( std::nullptr_t ) = delete; // (5)
```

All of these could be supported for `zstring_view`, however, all but (1) could be inadvertently misused. We propose still offering (2) as an O(1) constructor with checked precondition that the string indeed contains a null terminator after it, and none inside the string. We propose omitting the iterator (3) and range (4) constructors.

6.3 Member functions on `string_view` that return a `string_view`

In some cases the functions can be replicated on `zstring_view` with the return type being a `zstring_view`, while in some cases the return type loses the ability to guarantee null termination, and should still return a `string_view`. The types are somewhat entangled.

6.3.1 `substr`

The `substr` function is changed from a single function with a default argument, to two functions, one with one and one with two arguments. The one-argument `substr` always retains the end (identical to `string_view`'s `substr`) and returns a `zstring_view`; the two-argument `substr` at least in a conceptual sense chops off at least the null terminator from the end, and should always return a `string_view`.

6.3.2 `remove_prefix`

`remove_prefix` drops characters from the front of the string it's called on. Behavior fits.

6.3.3 `remove_suffix`

`remove_suffix` changes the string in-situ, and as such is unimplementable on zstring_view. It should be a function marked as =delete with a reason pointing users to use the two-argument `substr` function instead, which returns a `string_view`.

7 Bikeshedding

Null-terminated string view types are typically named `zstring_view` (N3921, P3081, GSL) or `cstring_view` (P1402). `cstring_view` follows the `std::string::c_str()` nomenclature while `zstring_view` has some establishment and recognizability.

Github code search shows similar popularity between https://github.com/search?q=%2F%5Cbcstring_view%5Cb%2F%20language%3Ac%2B%2B%20-is%3Afork&type=code (1.2k results as of the time of writing) and https://github.com/search?q=%2F%5Cbzstring_view%5Cb%2F+language%3Ac%2B%2B+-is%3Afork&type=code (680 results as of the time of writing). In refreshing this paper, it appears that cstring_view has gained ~200 added results, while zstring_view only added 8, hinting at a popular preference for the former name.

8 Reference Implementation

A reference implementation is at https://github.com/jeremy-rifkin/zstring_view.

9 Standard Library Changes

There are a handful of interfaces in the standard library which take `const string&` and really want a `zstring_view`

or possibly a `string_view`. These include:

- Constructors for stdexcept types, `system_error`, `format_error`, `ios_base::failure`, and `std::filesystem::filesystem_error`
- The `stoi` family of functions
- `random_device::random_device`
- Lots of locale interfaces
- `basic_filebuf::open`, `basic_ifstream::basic_ifstream`, `basic_ifstream::open`, `basic_ofstream::basic_ofstream`, `basic_ofstream::open`, `basic_fstream::basic_fstream`, and `basic_fstream::open`

There are further interfaces which take `const char*` and could have overloads taking a `zstring_view` or `string_view`.

We do not propose any such changes in this paper but would like to in a follow-up paper.

10 Polls

The first question to LEWG is, do we want to have the zstring_view type in the standard library? So far it looks like voting against the type leads to people implementing their own, including C++ committee members "implementing" their own zstring_view in wording. We also have a few direction questions with regards to either "fixing" string interface, or keeping the interfaces synchronized.

10.1 We encourage further work on the zstring_view proposal for the C++29 timeframe.

If so, we will return at the next meeting (Kona 2025) with the wording worked out and checked with a Core representative, with the intent of merging zstring_view early in the C++29 cycle. If not, we will retire this paper.

10.2 We prefer the name `cstring_view`

The bikeshedding mentioned above. At this point we can still choose which name to use. The authors have no preference.

- `cstring_view` is more common on Github at the time of writing, at a factor of about 2:1
- `cstring_view` matches existing `std::string` interface nomenclature (`c_str()`).
- `zstring_view` is the most recently proposed name in https://wg21.link/p3081

10.3 We prefer zstring_view to remove char_traits from its interface.

SG16 has long held that char_traits offer no value and are sometimes abused. If possible they would propose to remove it altogether P3681. This paper however is trying to add `zstring_view` to the standard without changing the direction for char_traits, and as such it is not proposing to remove char_traits.

Removing it anyway would necessarily lose information on implicit conversion from std::string through std::zstring_view to std::string_view, as the char_traits that were present on the string are no longer possible to transfer to the string_view. There was a discussion in SG16 with regards to char_traits needing to be removed, needing to be kept, or whether no objection existed to retaining it for compatibility, and the following poll was taken:

> Poll 1: P3655R0: No objection to use of std::char_traits for consistency and compatibility with std::string_view.
>
> | SF | F | N | A | SA |
> |---|---|---|---|---|
> | 7 | 1 | 0 | 0 | 0 |
>
> Strong consensus. Attendees: 8 (no abstentions)

The paper authors hold no strong opinion on whether char_traits should be kept or removed. Pragmatically we propose to keep char_traits to make the conversion through zstring_view lossless, and to treat removal of char_traits support as a uniform thing to be applied to all string types. P3681 disagrees, however, and we hope to see it up for discussion to reduce user-visible churn.

10.4 We prefer the zstring_view interface to add an array constructor

```
template <size_t N>
consteval basic_zstring_view(const charT (&str)[N]);
```

The advantage of adding this function is that it is a clean constructor to convert from a string literal with compile-time known size. The downsides are that it will create an interface difference between `string_view` and `zstring_view`.

10.5 We prefer to disallow contained NULs

String and string_view both support having a string with embedded NUL characters. On one hand it makes complete sense to do the same for zstring_view. On the other hand, the value of zstring_view lies in knowing it has a null terminator at `size()`, and while allowing embedded NULs would not strictly violate that - there still is a NUL terminator there - it would logically break it, as there would be a NUL before that that effectively terminates the string.

This gives rise to the possibility for bugs where strings are being used by their `strlen(x.c_str())` length, and elsewhere by their `x.size()`. These things are likely enough to have https://cwe.mitre.org/data/definitions/135.html and https://cwe.mitre.org/data/definitions/179.html, quote:

> An attacker could include dangerous input that bypasses validation protection mechanisms which can be used to launch various attacks including injection attacks, execute arbitrary code or cause other unintended behavior.

Consider the following call:

```
std::zstring_view user_command = "ls\0rm -rf /";
validate_command(user_command.c_str()); // older validation function that does its own strlen()
evaluate_command(user_command); // newer function that uses user_command.size()
```

We can prevent these bugs at this stage by disallowing contained NULs in the NUL-terminated string view type. But it is a breaking change to existing expectation.

11 Wording

This wording section is currently mostly high-level and synopsis changes. We defer full wording until feedback from LEWG.

Borrowing extensively from existing `string_view` wording.

Add to `[format.formatter.spec]` 2.2:

```
template<class traits>
  struct formatter<basic_zstring_view<charT, traits>, charT>;
```

Add to [format.formatter.spec] 4.1:

```
template<class traits>
  struct formatter<basic_zstring_view<char, traits>, wchar_t>;
```

Update string.view.general note 1 to mention conversion from `basic_zstring_view` to `basic_string_view`.

Add to basic.string.general:

```
    constexpr operator basic_zstring_view<charT, traits>() const noexcept;
```

Wording on additions deferred until a later revision.

§ 11.1 Zstring View Classes [zstring.view]

§ 11.1.1 General [zstring.view.general]

The class template basic_zstring_view describes an object that can refer to a constant contiguous sequence of char-like (strings.general) objects with the first element of the sequence at position zero. In the rest of string.view, the type of the char-like objects held in a `basic_string_view` object is designated by `charT`.

In all cases, `[data(), data() + size()]` is a valid range and `data() + size()` points at an object with value `charT()` (a "null terminator").

[Note 1: `basic_zstring_view<charT, ...>` is primarily useful when working with C APIs which expect null terminated strings. - end note]

§ 11.1.1.1 Header `` synopsis [zstring.view.synop]

```
// mostly freestanding
#include <compare>              // see [compare.syn]

namespace std {
  // [zstring.view.template], class template basic_zstring_view
  template<class charT, class traits = char_traits<charT>>
  class basic_zstring_view;                                          // partially freestanding

  template<class charT, class traits>
    constexpr bool ranges::enable_view<basic_zstring_view<charT, traits>> = true;
  template<class charT, class traits>
    constexpr bool ranges::enable_borrowed_range<basic_zstring_view<charT, traits>> = true;

  // [zstring.view.comparison], non-member comparison functions
  template<class charT, class traits>
    constexpr bool operator==(basic_zstring_view<charT, traits> x,
                              type_identity_t<basic_zstring_view<charT, traits>> y) noexcept;
  template<class charT, class traits>
    constexpr /* see below */
 operator<=>(basic_zstring_view<charT, traits> x,
                              type_identity_t<basic_zstring_view<charT, traits>> y) noexcept;

  // [zstring.view.io], inserters and extractors
  template<class charT, class traits>
    basic_ostream<charT, traits>&
      operator<<(basic_ostream<charT, traits>& os,
                 basic_zstring_view<charT, traits> str);                 // hosted

  // basic_zstring_view typedef-names
  using zstring_view    = basic_zstring_view<char>;
  using u8zstring_view  = basic_zstring_view<char8_t>;
  using u16zstring_view = basic_zstring_view<char16_t>;
  using u32zstring_view = basic_zstring_view<char32_t>;
  using wzstring_view   = basic_zstring_view<wchar_t>;

  // [zstring.view.hash], hash support
  template<class T> struct hash;
  template<> struct hash<zstring_view>;
```

```
  template<> struct hash<u8zstring_view>;
  template<> struct hash<u16zstring_view>;
  template<> struct hash<u32zstring_view>;
  template<> struct hash<wzstring_view>;

  inline namespace literals {
    inline namespace zstring_view_literals {
      // [zstring.view.literals], suffix for basic_zstring_view literals
      constexpr zstring_view    operator""zsv(const char* str, size_t len) noexcept;
      constexpr u8zstring_view  operator""zsv(const char8_t* str, size_t len) noexcept;
      constexpr u16zstring_view operator""zsv(const char16_t* str, size_t len) noexcept;
      constexpr u32zstring_view operator""zsv(const char32_t* str, size_t len) noexcept;
      constexpr wzstring_view   operator""zsv(const wchar_t* str, size_t len) noexcept;
    }
  }
}
```

§ 11.1.2 Class template basic_zstring_view [zstring.view.template]

§ 11.1.2.1 General [zstring.view.template.general]

```
namespace std {
  template<class charT, class traits = char_traits<charT>>
  class basic_zstring_view {
  public:
    // types
    using traits_type            = traits;
    using value_type             = charT;
    using pointer                = value_type*;
    using const_pointer          = const value_type*;
    using reference              = value_type&;
    using const_reference        = const value_type&;
    using const_iterator         = implementation-defined; // see [zstring.view.iterators]
    using iterator               = const_iterator;
    using const_reverse_iterator = reverse_iterator<const_iterator>;
    using reverse_iterator       = const_reverse_iterator;
    using size_type              = size_t;
    using difference_type        = ptrdiff_t;
    static constexpr size_type npos = size_type(-1);

    // [zstring.view.cons], construction and assignment
    constexpr basic_zstring_view() noexcept;
    basic_zstring_view(const basic_zstring_view&) noexcept = default;
    basic_zstring_view& operator=(const basic_zstring_view&) noexcept = default;
    constexpr basic_zstring_view(const charT* str);
    constexpr basic_zstring_view(const charT* str, size_type len);
    basic_zstring_view(nullptr_t) = delete;

    // [zstring.view.iterators], iterator support
    constexpr const_iterator begin() const noexcept;
    constexpr const_iterator end() const noexcept;
    constexpr const_iterator cbegin() const noexcept;
    constexpr const_iterator cend() const noexcept;
    constexpr const_reverse_iterator rbegin() const noexcept;
    constexpr const_reverse_iterator rend() const noexcept;
    constexpr const_reverse_iterator crbegin() const noexcept;
    constexpr const_reverse_iterator crend() const noexcept;

    // [zstring.view.capacity], capacity
    constexpr size_type size() const noexcept;
    constexpr size_type length() const noexcept;
    constexpr size_type max_size() const noexcept;
    [[nodiscard]] constexpr bool empty() const noexcept;

    // [zstring.view.access], element access
    constexpr const_reference operator[](size_type pos) const;
    constexpr const_reference at(size_type pos) const;
    constexpr const_reference front() const;
```

```
  constexpr const_reference back() const;
  constexpr const_pointer data() const noexcept;
  constexpr const_pointer c_str() const noexcept;

  operator basic_string_view<charT, traits>() const noexcept;

  // [zstring.view.modifiers], modifiers
  constexpr void remove_prefix(size_type n);
  constexpr void remove_suffix(size_type n) = delete("cannot remove_suffix in-place on zstring_view while
  constexpr void swap(basic_zstring_view& s) noexcept;

  // [zstring.view.ops], zstring operations
  constexpr size_type copy(charT* s, size_type n, size_type pos = 0) const;

  constexpr basic_string_view<charT, traits> substr(size_type pos = 0, size_type n = npos) const;

  constexpr int compare(basic_string_view<charT, traits> s) const noexcept;
  constexpr int compare(size_type pos1, size_type n1, basic_zstring_view s) const;
  constexpr int compare(size_type pos1, size_type n1, basic_zstring_view s,
                        size_type pos2, size_type n2) const;
  constexpr int compare(const charT* s) const;
  constexpr int compare(size_type pos1, size_type n1, const charT* s) const;
  constexpr int compare(size_type pos1, size_type n1, const charT* s, size_type n2) const;

  constexpr bool starts_with(basic_string_view<charT, traits> x) const noexcept;
  constexpr bool starts_with(charT x) const noexcept;
  constexpr bool starts_with(const charT* x) const;
  constexpr bool ends_with(basic_string_view<charT, traits> x) const noexcept;
  constexpr bool ends_with(charT x) const noexcept;
  constexpr bool ends_with(const charT* x) const;

  constexpr bool contains(basic_string_view<charT, traits> x) const noexcept;
  constexpr bool contains(charT x) const noexcept;
  constexpr bool contains(const charT* x) const;

  // [zstring.view.find], searching
  constexpr size_type find(basic_string_view<charT, traits> s, size_type pos = 0) const noexcept;
  constexpr size_type find(charT c, size_type pos = 0) const noexcept;
  constexpr size_type find(const charT* s, size_type pos, size_type n) const;
  constexpr size_type find(const charT* s, size_type pos = 0) const;
  constexpr size_type rfind(basic_string_view<charT, traits> s, size_type pos = npos) const noexcept;
  constexpr size_type rfind(charT c, size_type pos = npos) const noexcept;
  constexpr size_type rfind(const charT* s, size_type pos, size_type n) const;
  constexpr size_type rfind(const charT* s, size_type pos = npos) const;

  constexpr size_type find_first_of(basic_string_view<charT, traits> s, size_type pos = 0) const noexcept;
  constexpr size_type find_first_of(charT c, size_type pos = 0) const noexcept;
  constexpr size_type find_first_of(const charT* s, size_type pos, size_type n) const;
  constexpr size_type find_first_of(const charT* s, size_type pos = 0) const;
  constexpr size_type find_last_of(basic_string_view<charT, traits> s, size_type pos = npos) const noexcep
  constexpr size_type find_last_of(charT c, size_type pos = npos) const noexcept;
  constexpr size_type find_last_of(const charT* s, size_type pos, size_type n) const;
  constexpr size_type find_last_of(const charT* s, size_type pos = npos) const;
  constexpr size_type find_first_not_of(basic_string_view<charT, traits> s, size_type pos = 0) const noexc
  constexpr size_type find_first_not_of(charT c, size_type pos = 0) const noexcept;
  constexpr size_type find_first_not_of(const charT* s, size_type pos,
                                        size_type n) const;
  constexpr size_type find_first_not_of(const charT* s, size_type pos = 0) const;
  constexpr size_type find_last_not_of(basic_string_view<charT, traits> s,
                                       size_type pos = npos) const noexcept;
  constexpr size_type find_last_not_of(charT c, size_type pos = npos) const noexcept;
  constexpr size_type find_last_not_of(const charT* s, size_type pos,
                                       size_type n) const;
  constexpr size_type find_last_not_of(const charT* s, size_type pos = npos) const;

private:
  const_pointer data_;        // exposition only
  size_type size_;            // exposition only
```

```
  };
}
```

Wording on members deferred until a later revision.